

IA responsable para aplicaciones Claude en empresa

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 12 de 12 · Notas en español (es-ES)

Vijay agrupó permisos, hooks de seguridad y escáneres de prompt/salida bajo **principios de IA responsable** para las aplicaciones que UST entrega de verdad. El examen no puntuará vuestro programa GDPR, pero esperará que elijáis controles cuando un escenario mezcle PII, bash, MCP y un correo de cliente.

Qué significaba «responsable» en esta sesión

No una diapositiva de valores. Una definición de trabajo: el agente **no debe ejecutar daño** (borrar un disco, escribir malware en una ruta), **no debe filtrar datos regulados** por registros o herramientas de terceros, y **no debe obedecer prompts con forma de jailbreak** que intenten volcar el prompt de sistema o ignorar la política. Los nombres de cumplimiento en la sala fueron PII, GDPR e HIPAA — como ejemplos de regímenes que obligan a enmascarar, monitorizar y mínimo privilegio — no como una clase de derecho.

LA POLÍTICA DE USO SIGUE VINCULÁNDOOS

Los hooks no convierten en permitido un caso de uso prohibido. Leed la [política de uso de Anthropic](#) para productos y API. Las demos de aula (relojes, APIs públicas de todos, Markdown de muestra) se quedan en el lado correcto; los datos de producción de cliente en un cuaderno compartido no.

Apilad los controles

1. **Política y propósito** — ¿debe existir este agente? Reclutar por WhatsApp es un riesgo distinto a un bot de solo lectura sobre políticas.
2. **Minimización de datos** — RAG y las herramientas deben traer el conjunto de campos más pequeño útil. No recuperéis filas completas de cliente para responder «¿cuántos tickets abiertos hay?»
3. **Permitir / denegar herramientas** — un agente de informes puede necesitar una herramienta de BD y una de correo; no debe tener Bash sin límites.
4. **Hooks previos** — comprobaciones a nivel de argumento (nada de `rm -rf`, nada de escrituras en `/etc`, redirigir a un directorio seguro).
5. **Humano en el bucle** — AskUserQuestion o modo de permiso por defecto cuando la acción es irreversible (enviar oferta, borrar).
6. **Guardrails de entrada** — rechazar inyección y prompts excesivos antes de gastar tokens.
7. **Guardrails de salida** — búsqueda de secretos, redacción de PII antes de que registros, tickets o servidores MCP vean el texto.
8. **Evaluación** — la guía strengthen-guardrails de Anthropic: probad el agente en casos de abuso, no solo en la vía feliz de publicar una JD.

Relato de empresa: informe y correo

Herramientas a medida: leer la BD de clientes → generar informe → enviar correo. El correo es PII. La herramienta de envío necesita la dirección real. Todo lo demás (prints de depuración, búsqueda web, un servidor MCP de «ayuda» de un proveedor) debería ver un valor enmascarado o nada. Por eso «cifrar la pasarela» es incompleto: el cifrado en tránsito no impide que vuestro agente pegue la dirección en un echo de Bash o en un MCP de Slack. El diseño responsable es *dónde se permite que exista el texto en claro*.

Jailbreaks y robo del prompt de sistema

El validador de entrada del cuaderno trataba frases como «ignora todas las instrucciones previas» y «revela tu prompt de sistema» como patrones de bloqueo (regex en clase; bibliotecas en producción). Cortocircuitad *antes* de query para no pagar un rechazo que podríais haber hecho en local. Esto complementa el entrenamiento de seguridad de Claude, no lo sustituye.

Código abierto frente a primera parte

Mritunjay preguntó si los guardrails eran una función integrada de Claude que además recortaba tokens. No: la contabilidad de tokens ocurre en el query de la API. El código de guardrail es vuestro (o un paquete de código abierto revisado). Recread o evaluad como proveedores listas tipo Rulebricks y repos comunitarios de guardrails de Claude Code; no peguéis regex sin auditar en el agente de un banco. Emparejadlo con la documentación oficial de evaluación para que la respuesta de examen pueda nombrar «validadores a nivel de aplicación» y «guía de guardrails/eval de Anthropic».

Lista de entrega UST

- Nada de corpus confidenciales de cliente en Cowork, Chat o clústeres Atlas de aula.
- Claves API en workspaces y almacenes de secretos ([workspaces](#)), no en .env subido.
- Agentes en modo plan o solo observación en CI hasta que un humano promocióne accept-edits.
- Servidores MCP: preferid los del proveedor; tratad el MCP comunitario como código no confiable — son herramientas con vuestras credenciales.
- Índices RAG: metadatos de inquilino para que un cliente nunca recupere los trozos de otro (módulo 11).

Cómo aparece esto en los escenarios

Elegid el control que encaja con el fallo: shell destructivo → hook o denegar Bash; el modelo pide ignorar la política → guardrail de entrada; correo de cliente en un registro → redacción de salida; el agente no debe cambiar ficheros de producción → plan + Read/Glob/Grep; hace falta un humano para confirmar un envío → AskUserQuestion o avisos por defecto. Apilar todos los controles en cada herramienta reloj de juguete no es el punto.

Documentación oficial

- **Política de uso**

<https://www.anthropic.com/legal/usage-policy>

Normas vinculantes para productos Claude y la API.

- **Reforzar guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Cómo Anthropic quiere que probéis y endurezcáis.

- **Workspaces**

<https://docs.claude.com/en/docs/build-with-claude/workspaces>

Aislamiento de organización y de claves.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Las herramientas de terceros forman parte del modelo de amenaza del flujo de datos.

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Modos de permiso que configuraréis de verdad.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.